Universidad Pascual Bravo

Facultad de Ingeniería

Programa: Tecnología en Desarrollo de Software

Asignatura: Bases de datos I G(101)

Docente: Oralia Cortes Grajales

S1. Tarea. Implementación de la estructura completa de una base de datos (DDL)

Estudiante: Juan Esteban Villada Ruiz

C.C 1039049872

Medellín – Colombia

Octubre 18 de 2026

Identificación y Justificación del SGBD

Para la implementación física del modelo de datos de la Red Social Pascualina, seleccioné PostgreSQL (gestionado a través de pgAdmin 4) como el Sistema Gestor de Bases de Datos (SGBD) relacional de grado empresarial. La elección se fundamenta en un análisis comparativo frente a otras alternativas del mercado como MySQL y SQLite, evaluando criterios de consistencia, concurrencia y soporte del estándar SQL ANSI.

Ventajas Competitivas de PostgreSQL

Cumplimiento ACID Riguroso: Garantiza el cumplimiento estricto del estándar ACID (Atomicidad, Consistencia, Aislamiento y Durabilidad) bajo el mecanismo MVCC (Multi-Version Concurrency Control), evitando bloqueos de lectura durante escrituras masivas en tablas de alta concurrencia como PUBLICACION y REACCION.

Estándar DDL Completo: Ofrece soporte nativo para claves primarias compuestas, secuencias automatizadas (IDENTITY / SERIAL), restricciones de dominio (CHECK), llaves foráneas con acciones referenciales en cascada y tipos de datos enriquecidos como TIMESTAMP WITH TIME ZONE y TEXT.

Soporte de Atributos Extensos (TOAST): A diferencia de SQLite o versiones básicas de MySQL, PostgreSQL administra óptimamente el almacenamiento en disco de campos extensos sin longitud fija (como el contenido de publicaciones o recursos académicos) mediante la técnica TOAST (The Oversized-Attribute Storage Technique).

Cuadro Comparativo de Opciones Evaluadas

SGBD	Ventajas Clave	Desventajas / Limitaciones	Decisión de Selección
PostgreSQL	MVCC, control de concurrencia impecable, extensiones avanzadas, estricto en reglas de integridad.	Consumo ligeramente mayor de memoria RAM en instancias iniciales.	**SELECCIONADO**
MySQL / MariaDB	Ampliamente difundido, rápida lectura en consultas simples.	Sintaxis DDL menos flexible para ciertas restricciones compuestas y variaciones en comportamiento de claves foráneas.	**Descartado**
SQLite	Cero configuración, ligero, basado en archivo único.	Sin gestión de concurrencia multiusuario nativa ni control granular de permisos.	**Descartado**

Transformación del Modelo Lógico al Modelo Físico

El paso del modelo lógico (15 tablas normalizadas en 3FN) al modelo físico funcional implicó traducir las entidades, atributos, cardinalidades y reglas de negocio a estructuras nativas de PostgreSQL. Los criterios de transformación aplicados fueron los siguientes:

Nomenclatura y Estándares: Las entidades del modelo conceptual/lógico se tradujeron a sentencias CREATE TABLE utilizando nomenclatura en minúsculas (snake_case) según las recomendaciones oficiales de PostgreSQL para evitar el uso obligado de comillas dobles en las consultas SQL.

Mapeo Estricto de Atributos: Cada atributo se mapeó a un tipo de dato nativo del SGBD. Se evitó el tipo genérico TEXT para atributos de longitud predecible (como

nombres, correos o nombres de usuario), aplicando VARCHAR con límites óptimos para minimizar el desperdicio de espacio e imponer validación desde el motor.

Resolución de Relaciones N:M: Se convirtieron en tablas puente intermedias (como estudiante_interes, integrante_grupo y miembro_equipo) con llaves primarias compuestas que evitan registros duplicados.

Manejo de Integridad Referencial: Se implementaron restricciones FOREIGN KEY con cláusulas explícitas ON DELETE CASCADE y ON UPDATE CASCADE para mantener la consistencia referencial automatizada en todo el grafo de datos.

Inventario Completo de Tablas y Justificación Técnica

A continuación se presenta la descripción detallada de las 15 tablas que componen el modelo físico de la Red Social Pascualina, justificando las decisiones de diseño para cada una:

1. ESTUDIANTE (Tabla Padre Principal)

Almacena los datos personales e institucionales de los usuarios registrados.

- **id_estudiante:** INT / IDENTITY — PK autoincremental de 4 bytes. Se prefiere sobre BIGINT porque la población estudiantil no sobrepasará los 2,147 millones de registros.
- **nombre / apellido:** VARCHAR(50) — Longitud acotada suficiente para nombres reales, evitando desperdicio de almacenamiento.
- **correo:** VARCHAR(100) UNIQUE — Restricción UNIQUE para evitar duplicidad de cuentas institucionales.
- **carrera / semestre:** VARCHAR(100) / INT — Semestre incluye un CHECK (semestre BETWEEN 1 AND 12) para validar el rango de la carrera.
- **fecha_registro:** TIMESTAMP — Sustituye DATETIME registrando la fecha y hora exacta de alta con DEFAULT CURRENT_TIMESTAMP.

2. INTERES (Tabla Catálogo)

Catálogo centralizado de gustos, temas de estudio y aficiones.

- **id_interes:** INT / IDENTITY — PK entero ligero para indexación rápida.
- **nombre_interes:** VARCHAR(50) UNIQUE — Garantiza que no existan etiquetas de intereses duplicadas en la plataforma.

3. ESTUDIANTE_INTERES (Tabla Puente N:M)

Relaciona los estudiantes con sus múltiples intereses académicos y personales.

- **id_estudiante / id_interes:** INT — Forman una Llave Primaria Compuesta (id_estudiante, id_interes), impidiendo que un estudiante asocie dos veces el mismo interés y asegurando 2FN.

4. PUBLICACION (Tabla Hija / Relación 1:N)

Almacena los mensajes, recursos y dudas compartidas por los estudiantes.

- **id_publicacion:** INT / IDENTITY — PK autoincremental de la publicación.
- **id_estudiante:** INT — FK referenciada a ESTUDIANTE(id_estudiante) con ON DELETE CASCADE.
- **contenido:** TEXT — Se elige TEXT en lugar de VARCHAR porque la longitud de un post es altamente variable y puede ser muy extensa.

5. COMENTARIO (Tabla Hija 1:N)

Respuestas y discusiones asociadas a las publicaciones.

- **id_comentario:** INT / IDENTITY — PK propia. Se descartó una PK compuesta porque un estudiante puede comentar varias veces en el mismo post.
- **contenido:** TEXT — Permite explicaciones y argumentos amplios en los hilos de debate.

6. REACCION (Tabla Interacción)

Interacciones rápidas (Me gusta, Me sirve, etc.) en las publicaciones.

- **id_reaccion:** INT / IDENTITY — PK simple.
- **tipo:** VARCHAR(30) — Etiqueta corta de reacción.
- **(id_estudiante, id_publicacion):** UNIQUE — Restricción UNIQUE compuesta que impide que un usuario reaccione más de una vez a la misma publicación.

7. GRUPO_ESTUDIO (Tabla Entidad)

Grupos o comunidades formadas por materias o intereses.

- **id_grupo:** INT / IDENTITY — PK autoincremental.
- **nombre_grupo:** VARCHAR(100) — Nombre distintivo del grupo de estudio.
- **descripcion:** TEXT — Información detallada de los objetivos del grupo.

8. INTEGRANTE_GRUPO (Tabla Puente N:M)

Asociación de estudiantes a grupos de estudio con roles.

- **id_estudiante / id_grupo:** INT — PK compuesta para impedir membresías duplicadas.
- **rol:** VARCHAR(30) — Define si es 'Líder', 'Moderador' o 'Miembro'.

9. EVENTO (Tabla Entidad)

Eventos académicos, hackatones o talleres programados.

- **id_evento:** INT / IDENTITY — PK del evento.
- **titulo:** VARCHAR(150) — Título representativo.
- **fecha_inicio / fecha_fin:** TIMESTAMP — Control temporal estricto del evento.

10. EQUIPO (Tabla Entidad)

Equipos de trabajo organizados .

- **id_equipo:** INT / IDENTITY — PK única del equipo.
- **id_evento:** INT — FK vinculada a la hackathon correspondiente.

11. MIEMBRO_EQUIPO (Tabla Puente N:M)

Estudiantes integrantes de un equipo .

- **id_estudiante / id_equipo:** INT — PK compuesta que garantiza un solo registro por estudiante en cada equipo.

12. MENTORIA (Tabla Entidad)

Sesiones de asesoría académica entre estudiantes avanzados y principiantes.

- **id_mentoria:** INT / IDENTITY — PK de la mentoría.
- **id_mentor / id_estudiante:** INT — Dos FKs apuntando a ESTUDIANTE(id_estudiante), diferenciando el rol del mentor y del aprendiz.
- **materia:** VARCHAR(100) — Asignatura o tema abordado.

13. RECURSO_ACADEMICO (Tabla Entidad)

Documentos, guías o enlaces compartidos.

- **id_recurso:** INT / IDENTITY — PK del recurso.
- **url_archivo:** VARCHAR(255) — Ruta o enlace donde se almacena el recurso digital.

14. MENSAJE_DIRECTO (Tabla Comunicación)

Mensajería privada entre estudiantes.

- **id_mensaje:** INT / IDENTITY — PK del mensaje.
- **id_emisor / id_receptor:** INT — FKs cruzadas hacia ESTUDIANTE(id_estudiante) para gestionar la conversación bilateral.
- **contenido:** TEXT — Texto del mensaje privado.

15. NOTIFICACION (Tabla Sistema)

Alertas y avisos del sistema para cada estudiante.

- **id_notificacion:** INT / IDENTITY — PK única.

- **leido:** BOOLEAN — Valor booleano (DEFAULT FALSE) para control de lectura del usuario.

Implementación de Relaciones e Integridad Referencial

Toda la coherencia estructural de la Red Social Pascualina está protegida a nivel de base de datos mediante la configuración explícita de reglas referenciales `FOREIGN KEY`. A continuación se detallan las relaciones más relevantes y la justificación de sus acciones:

ESTUDIANTE (Padre) ➔ PUBLICACION (Hija)

- **Acciones Referenciales:** ON DELETE CASCADE / ON UPDATE CASCADE
- **Justificación Técnica:** Si un estudiante borra su cuenta, se borran automáticamente todas sus publicaciones. Esto previene registros huérfanos sin autor en el muro.

PUBLICACION (Padre) ➔ COMENTARIO (Hija)

- **Acciones Referenciales:** ON DELETE CASCADE / ON UPDATE CASCADE
- **Justificación Técnica:** Si una publicación es eliminada por el autor o moderación, sus comentarios pierden su contexto y se borran en cascada.

PUBLICACION (Padre) ➔ REACCION (Hija)

- **Acciones Referenciales:** ON DELETE CASCADE / ON UPDATE CASCADE
- **Justificación Técnica:** Las interacciones dependen directamente de la existencia del post; si el post desaparece, la reacción carece de objeto.

ESTUDIANTE / INTERES ➔ ESTUDIANTE_INTERES

- **Acciones Referenciales:** ON DELETE CASCADE / ON UPDATE CASCADE
- **Justificación Técnica:** Al eliminar un interés del catálogo o dar de baja a un usuario, la tabla intermedia limpia automáticamente las asociaciones.

GRUPO_ESTUDIO ➔ INTEGRANTE_GRUPO

- **Acciones Referenciales:** ON DELETE CASCADE / ON UPDATE CASCADE
- **Justificación Técnica:** Al disolverse un grupo de estudio, se eliminan los registros de membresía correspondientes.

5. Script DDL Completo en PostgreSQL (`script_red_social.sql`)

A continuación se transcribe el código fuente DDL completo que debe ser ejecutado en PostgreSQL / pgAdmin 4. Este script crea la base de datos, limpia las tablas existentes en orden inverso de dependencias y construye la totalidad de las 15 tablas con sus restricciones:

Conclusiones e Impacto Individual de Aprendizaje

A nivel personal, la ejecución de este proyecto representó el cierre práctico fundamental donde los conceptos teóricos de modelado de datos cobraron vida en un entorno de desarrollo real con PostgreSQL.

Superación de Dificultades: Al principio del desarrollo, la mayor inquietud radicaba en garantizar que las acciones referenciales ON DELETE CASCADE no provocaran borrados indeseados en cadena. Comprender cómo se propaga la integridad referencial permitió diseñar el modelo con total confianza operacional.

Impacto Técnico de los Tipos de Datos: Entender la diferencia entre usar VARCHAR acotado para atributos fijos y TEXT para atributos extensos permitió dimensionar el impacto que tienen las decisiones DDL en el consumo de memoria del servidor y la velocidad de indexación.

Valor del Diseño Normalizado: La estructuración de llaves primarias compuestas en tablas puente (como estudiante_interes) demostró ser la mejor salvaguarda desde la arquitectura para garantizar el cumplimiento de la Segunda Forma Normal (2FN) sin depender de validaciones en el código de aplicación.

LINK VIDEO: Implementación de la estructura completa de una base de datos (DDL).